Steady Two-Dimensional Heat Conduction

A Second-Order Finite-Volume Solver with Iterative and Direct Methods

Arian Shahnoori

October 10, 2026

Abstract

A two-dimensional finite-volume solver is developed for the steady-state heat conduction equation on a square domain. The Laplace equation is discretized using a second-order cell-centered finite-volume formulation on a structured Cartesian grid. Four solution methods are implemented: Point Gauss-Seidel, Successive Over-Relaxation (SOR), Line-SOR, and a fully implicit direct method. The numerical solutions are verified through comparison with the analytical solution. Computational studies investigate convergence behavior, iteration counts, and CPU time across different relaxation factors and mesh resolutions. Post-processing techniques, including temperature contours, error distributions, line-cut comparisons, and residual convergence histories, are employed to assess numerical accuracy and compare the computational performance of the solution methods.

Problem	Steady Two-Dimensional Heat Conduction
Governing Equation	Laplace Equation
Discretization	Cell-Centered Finite Volume Method
Spatial Accuracy	Second Order
Solution Methods	Point Gauss-Seidel, SOR, Line-SOR, Direct
Programming Language	Python
Verification	Analytical Solution
Performance Evaluation	Convergence and CPU-Time Comparisons

Contents

1 Introduction **4**

2 Governing Equation **4**

3 Finite-Volume Discretization **5**

4 Numerical Solution Methods **6**

4.1 Boundary Condition Implementation 7

4.2 Residual Definition 7

4.3 Convergence Criterion 7

4.4 Point Gauss-Seidel Iteration 8

4.5 Successive Over-Relaxation (SOR) 8

4.6 Line-SOR 9

4.6.1 Line-SOR Formulation 9

4.6.2 Matrix Formulation 10

4.6.3 Temperature Update and Implementation 11

4.7 Fully Implicit Direct Method 11

4.7.1 Linear System Formulation 12

4.7.2 Coefficient Matrix Assembly 13

4.7.3 Direct Solution and Computational Time 13

5 Computational Implementation **14**

5.1 Software Architecture 14

5.2 Study Organization 15

5.3 Solution Algorithm 16

5.4 Solver Implementation 16

5.5 Grid Generation 17

5.6 Post-Processing and Performance Evaluation 17

6 Results and Discussion **18**

6.1 Temperature Distribution 18

6.2 Error Field 18

6.3 Convergence History 19

6.4 Line-Cut Comparison 20
6.5 SOR Convergence and Computational Cost 20
6.6 Line-SOR Convergence and Computational Cost 21
6.7 Fully Implicit Direct Solver 22
6.7.1 CPU-Time Comparison Across Mesh Sizes 22
6.7.2 Verification Against the Analytical Solution 24

7 Conclusion **25**

A Computational Setup **25**
A.1 Hardware Configuration 25
A.2 Software Environment 26
A.3 Timing Methodology 26

1. Introduction

Heat conduction is one of the three fundamental modes of heat transfer and plays a central role in numerous engineering applications, including thermal management systems, heat exchangers, electronic cooling, and energy systems. While analytical solutions exist for a limited number of idealized problems, most practical engineering applications involve complex geometries, material properties, or boundary conditions that require numerical solution techniques.

Before applying a numerical method to complex engineering problems, it is essential to verify that the numerical implementation correctly reproduces solutions for cases where an analytical solution is available. Code verification provides confidence that the governing equations, boundary conditions, spatial discretization, and numerical solution algorithms have been implemented correctly.

The objective of this project is to develop a numerical solver for the two-dimensional steady-state heat conduction equation on a square domain. The governing equation is discretized using a second-order finite-volume formulation on a structured Cartesian mesh and solved iteratively using the Point Gauss-Seidel method. The numerical solution is verified through direct comparison with the corresponding analytical solution of the Laplace equation.

Beyond validating the numerical implementation, this project establishes a reusable computational framework for future developments, including mesh refinement studies, accelerated iterative methods such as Successive Over-Relaxation (SOR), and more advanced computational fluid dynamics applications.

2. Governing Equation

The physical problem considered in this study is two-dimensional steady-state heat conduction in a square domain with no internal heat generation. Under these assumptions, the conservation of energy reduces to the Laplace equation,

$$\nabla^2 T = \frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0 \tag{1}$$

where T denotes the temperature field and x and y are the Cartesian spatial coordinates.

The computational domain and the prescribed boundary conditions are illustrated in Figure 1. Dirichlet boundary conditions are imposed on all boundaries, meaning that the temperature is explicitly prescribed along the boundaries of the computational domain. These boundary conditions uniquely determine the solution of Equation 1.

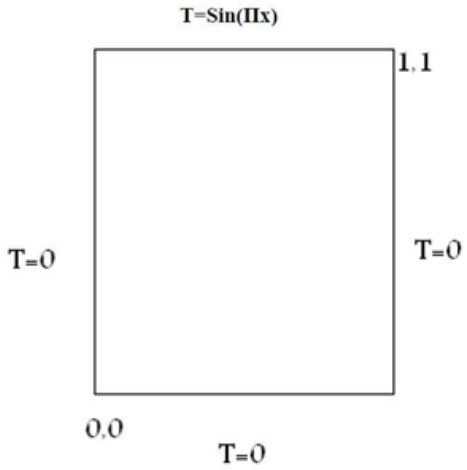


Figure 1: Computational domain and prescribed boundary conditions.

For the selected boundary conditions, an analytical solution exists and can therefore be used to verify the numerical implementation. The exact solution of Equation 1 is

$$T(x, y) = \frac{\sin(\pi x)\sinh(\pi y)}{\sinh(\pi)} \tag{2}$$

The existence of the analytical solution given by Equation 2 makes this problem an ideal verification benchmark. Throughout this project, the numerical solution obtained using the finite-volume discretization and the Point Gauss-Seidel iterative solver is compared directly with the analytical temperature distribution to assess the correctness and accuracy of the numerical implementation.

3. Finite-Volume Discretization

The governing equation was discretized using the finite-volume method on a structured Cartesian mesh. In this approach, the computational domain is divided into a collection of non-overlapping control volumes, and the governing equation is enforced over each control volume individually. Unlike finite-difference methods, which approximate the governing equation directly at discrete points, the finite-volume method is derived from the integral form of the conservation law, thereby ensuring local conservation within every control volume.

Expressing the Laplace operator in divergence form and integrating over an arbitrary control volume yields

$$\iint_{CV} \nabla \cdot (\nabla T)\, dA = 0 \tag{3}$$

Applying the divergence theorem converts the volume integral into a surface integral over the control-volume boundary,

$$\oint_{CS} \nabla T \cdot \mathbf{n}\, ds = 0 \tag{4}$$

where **n** denotes the outward unit normal vector to the control surface. Equation 4 states that, at steady state and in the absence of internal heat generation, the net diffusive heat flux through the boundary of every control volume is zero.

The computational stencil and notation adopted throughout the discretization procedure are illustrated in Figure 2.

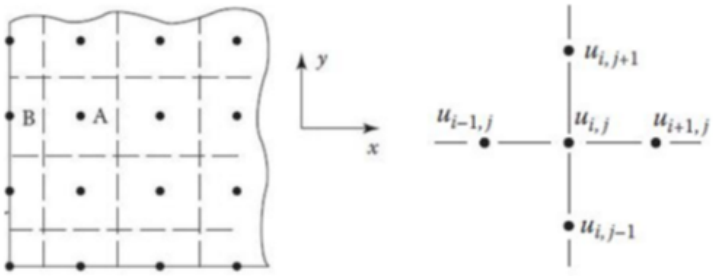


Figure 2: Computational stencil and control-volume notation used in the finite-volume discretization.

To achieve second-order spatial accuracy, the temperature gradients at the control-volume faces are approximated using central differences. For a uniform Cartesian mesh, the temperature gradient at the east face of a control volume is approximated by

$$\left.\frac{\partial T}{\partial x}\right|_{i+\frac{1}{2}} = \frac{T_{i+1} - T_i}{\Delta x} + \mathcal{O}(\Delta x^2) \tag{5}$$

An analogous expression is obtained for the remaining control-volume faces in both coordinate directions.

Substituting the face-gradient approximations into Equation 4 and evaluating the fluxes across the four faces of each control volume yields the discrete form of the governing equation,

$$\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta y^2} = 0 \tag{6}$$

For the present study, a uniform Cartesian mesh is employed such that

$$\Delta x = \Delta y.$$

Under this condition, Equation 6 provides a second-order accurate approximation of the continuous Laplace equation throughout the computational domain.

The resulting system of algebraic equations is coupled through the neighboring control-volume temperatures and therefore cannot be solved independently for each cell. An iterative solution procedure is consequently required to obtain the numerical temperature distribution.

4. Numerical Solution Methods

The finite-volume discretization presented in the previous section produces a coupled system of algebraic equations for the control-volume temperatures. This section first describes

the boundary-condition treatment and convergence measures, then presents four solution approaches: Point Gauss-Seidel, Successive Over-Relaxation (SOR), Line-SOR, and a direct solution of the assembled linear system.

4.1 Boundary Condition Implementation

The numerical formulation employs a cell-centered finite-volume mesh in which temperature values are stored at the centers of the control volumes. Since the physical boundaries do not coincide with computational nodes, the prescribed Dirichlet boundary conditions cannot be imposed directly.

Instead, an additional layer of fictitious control volumes, commonly referred to as ghost cells, is introduced outside the computational domain. Assuming a linear temperature variation between the neighboring cell centers, the ghost-cell value is obtained from

$$T_G = 2T_b - T_P \tag{7}$$

where T_b is the prescribed boundary temperature, T_P denotes the adjacent interior control-volume temperature, and T_G is the ghost-cell temperature.

Updating the ghost cells ensures that the prescribed boundary conditions are incorporated into the discrete governing equation, including for control volumes adjacent to the physical boundaries. The ghost-cell values are updated during the iterative solution process after each complete sweep of the computational domain.

4.2 Residual Definition

The convergence of the numerical solution is assessed by evaluating the residual of the discrete governing equation. The residual measures the extent to which the current temperature field satisfies the discretized equation.

For the present problem, where $\Delta x = \Delta y$, the residual associated with each control volume is

$$R_{i,j} = \frac{T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j}}{\Delta x^2} \tag{8}$$

As the numerical solution approaches the converged solution of the discrete system, the residual tends toward zero throughout the computational domain.

4.3 Convergence Criterion

Since each control volume has its own residual, convergence is monitored using a norm of the residual field. The general definition of the residual norm is

$$L_p = \left(\frac{\sum_{i=1}^{N_{\mathrm{CV}}} A_i |R_i|^p}{\sum_{i=1}^{N_{\mathrm{CV}}} A_i} \right)^{1/p} \tag{9}$$

where N_{CV} is the total number of control volumes, A_i is the area of control volume i, and R_i is

its corresponding residual.

Since all control volumes have identical area in the present structured mesh, the L_2 residual norm simplifies to

$$L_2 = \left(\frac{\sum_{i=1}^{N_{\text{CV}}} R_i^2}{N_{\text{CV}}} \right)^{1/2} \tag{10}$$

The iterative solution procedure continues until the L_2 residual norm falls below the prescribed convergence tolerance. The resulting temperature field is then used for post-processing and verification against the analytical solution.

4.4 Point Gauss-Seidel Iteration

The Point Gauss-Seidel method is employed as the baseline iterative solver. Rearranging Equation 6 to isolate the temperature at the current control volume yields the iterative update equation

$$\begin{aligned} T_{i,j}^{k+1} = & \frac{(\Delta y)^2}{2\left[(\Delta x)^2 + (\Delta y)^2\right]} \left(T_{i+1,j}^{k} + T_{i-1,j}^{k+1} \right) \\ & + \frac{(\Delta x)^2}{2\left[(\Delta x)^2 + (\Delta y)^2\right]} \left(T_{i,j+1}^{k} + T_{i,j-1}^{k+1} \right) \end{aligned} \tag{11}$$

where k denotes the iteration number.

Starting from an initial temperature distribution, the solution is updated sequentially throughout the computational domain. As the computation proceeds through the grid, newly computed temperatures immediately replace their previous values and are subsequently used when updating neighboring control volumes. This feature distinguishes the Gauss-Seidel method from the Jacobi method.

After each complete sweep, the ghost-cell values are updated according to Equation 7. The residual norm is then evaluated, and the process is repeated until the convergence criterion is satisfied.

4.5 Successive Over-Relaxation (SOR)

To investigate the effect of the iterative solution method on convergence, the Successive Over-Relaxation (SOR) method is applied using relaxation factors

$$\omega = 1.1,\ 1.3,\ 1.5.$$

The convergence behavior is evaluated in terms of both the number of iterations and the CPU time required to reach the prescribed residual tolerance. The results are compared with those obtained using the Point Gauss-Seidel method under the same mesh and convergence criterion.

The SOR method follows the same sequential update procedure as Gauss-Seidel but introduces a relaxation step. First, the Gauss-Seidel estimate, $T_{i,j}^{\text{GS}}$, is calculated using the most recently

available neighboring temperatures. The temperature is then updated according to

$$T_{i,j}^{k+1} = T_{i,j}^{k} + \omega \left(T_{i,j}^{\mathrm{GS}} - T_{i,j}^{k} \right). \tag{12}$$

Here, ω is the relaxation factor. Values of $\omega > 1$ correspond to over-relaxation and may accelerate convergence, whereas $0 < \omega < 1$ corresponds to under-relaxation. The effectiveness of over-relaxation depends on the problem and the selected relaxation factor; excessively large values may lead to poor convergence or divergence.

For the present study, a 40×40 control-volume mesh and a residual tolerance of 10^{-12} are used. The Point Gauss-Seidel method and SOR with $\omega = 1.1$, 1.3, and 1.5 are executed from the same initial condition. Residual histories are recorded as functions of both iteration number and CPU time. The resulting convergence behavior is presented in Figure 7, while the corresponding iteration counts and computational times are reported in Table 3.

4.6 Line-SOR

The Line Successive Over-Relaxation (Line-SOR) method is introduced to improve the convergence of the iterative solution. Unlike point-wise SOR, which updates one control volume at a time, Line-SOR determines the temperature corrections along an entire line of control volumes simultaneously. The method is therefore semi-implicit: the solution is treated implicitly in one spatial direction and explicitly in the other.

For the present problem, the y-direction is selected as the implicit direction, while the x-direction is treated explicitly. This choice couples vertically adjacent control volumes and incorporates their interaction with the upper boundary through the line system. The corrections along each vertical line are determined simultaneously, while the coupling in the horizontal direction is treated explicitly. The relaxation factor is set to

$$\omega = 1.1.$$

The solver is applied to the four mesh configurations with $N = 10$, 20, 30, and 40 control volumes in each direction. Its computational performance is compared with that of point-wise SOR using the same relaxation factor, mesh resolution, and convergence tolerance.

4.6.1 Line-SOR Formulation

The Point SOR method updates the temperature using a correction to the current value. In Line-SOR, instead of computing the corrections independently, the corrections along a complete line are determined simultaneously.

For a fixed value of i, the unknown corrections are

$$\delta T_{i,1}^{k+1}, \delta T_{i,2}^{k+1}, \ldots, \delta T_{i,N}^{k+1},$$

where k denotes the previous iteration and $k+1$ denotes the current iteration.

For a uniform grid, $\Delta x = \Delta y$, the discrete Laplace equation can be expressed in terms of the

corrections as

$$-\frac{1}{4}\delta T_{i,j+1}^{k+1} + \delta T_{i,j}^{k+1} - \frac{1}{4}\delta T_{i,j-1}^{k+1} = b_{i,j}, \tag{13}$$

where the right-hand-side term is

$$b_{i,j} = \frac{1}{4}\left(T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1}\right) - T_{i,j}. \tag{14}$$

The temperature values on the right-hand side are the currently available values during the sweep. In particular, the neighboring values in the explicit x-direction are taken from the current solution field, whereas the corrections in the implicit y-direction are treated as simultaneous unknowns.

Equations 13 and 14 form a tridiagonal system for the corrections along each vertical line.

4.6.2 Matrix Formulation

For a fixed i, the vector of unknown corrections is defined as

$$\boldsymbol{\delta T}_i = \begin{bmatrix} \delta T_{i,1}^{k+1} \\ \delta T_{i,2}^{k+1} \\ \vdots \\ \delta T_{i,N}^{k+1} \end{bmatrix}, \tag{15}$$

and the corresponding right-hand-side vector is

$$\mathbf{b}_i = \begin{bmatrix} b_{i,1} \\ b_{i,2} \\ \vdots \\ b_{i,N} \end{bmatrix}. \tag{16}$$

The line equations can then be assembled into the linear system

$$\mathbf{A}\boldsymbol{\delta T}_i = \mathbf{b}_i, \tag{17}$$

where the coefficient matrix is

$$\mathbf{A} = \begin{bmatrix} \frac{5}{4} & -\frac{1}{4} & 0 & \cdots & 0 \\ -\frac{1}{4} & 1 & -\frac{1}{4} & \ddots & \vdots \\ 0 & -\frac{1}{4} & 1 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -\frac{1}{4} \\ 0 & \cdots & 0 & -\frac{1}{4} & \frac{5}{4} \end{bmatrix}. \tag{18}$$

The first and last diagonal coefficients differ from those of the interior rows because of the Dirichlet boundary conditions implemented through ghost cells. At the lower boundary, the

ghost-cell relation gives

$$\delta T_{i,0}^{k+1} = -\delta T_{i,1}^{k+1},$$

while at the upper boundary,

$$\delta T_{i,N+1}^{k+1} = -\delta T_{i,N}^{k+1}.$$

Substituting these relations into the line equations changes the corresponding diagonal coefficients from 1 to $1 + \frac{1}{4} = \frac{5}{4}$.

Consequently, the first matrix row contains only the diagonal coefficient $\frac{5}{4}$ and the upper off-diagonal coefficient $-\frac{1}{4}$. Similarly, the last row contains the lower off-diagonal coefficient and the diagonal coefficient $\frac{5}{4}$. No coefficient is assigned outside the matrix at either end of the line.

4.6.3 Temperature Update and Implementation

After solving Equation 17, the temperature field is updated using the relaxation factor:

$$T_{i,j}^{k+1} = T_{i,j}^{k} + \omega \delta T_{i,j}^{k+1}. \tag{19}$$

The computational procedure proceeds through the vertical lines of the domain. For each line, the right-hand-side vector is constructed from the currently available temperature values, the line system is solved, and the resulting corrections are used to update the temperatures. The upper and lower ghost cells are then updated for that line. After all lines have been processed, the left and right ghost-cell values are updated.

The coefficient matrix depends only on the mesh resolution and is therefore constructed once and reused for every line and iteration. In the current implementation, the linear system is solved using NumPy's `numpy.linalg.solve()` function. Although the coefficient matrix is tridiagonal and could be solved using a specialized algorithm such as the Thomas algorithm, that optimization is not included in the present implementation.

After each complete sweep, the residual norm is evaluated according to Equation 10. Iterations continue until the prescribed convergence tolerance is satisfied. The computational times of Line-SOR and point-wise SOR are subsequently compared for the selected mesh sizes. The corresponding performance results are presented in Figure 8 and Table 4.

4.7 Fully Implicit Direct Method

Finally, the fully implicit direct method is employed to solve the discretized governing equation. Unlike the iterative methods described previously, this approach assembles the equations for all control volumes into a single linear system and solves for the complete temperature field simultaneously.

The method is applied to mesh sizes $N = 10$, 20, 30, and 40. The computational time is recorded for each case and compared with the corresponding results obtained using the iterative solvers.

4.7.1 Linear System Formulation

In the fully implicit direct method, the discretized equations for all control volumes are assembled into the linear system

$$\mathbf{AX} = \mathbf{d}, \tag{20}$$

where $\mathbf{A}$ is the coefficient matrix, $\mathbf{X}$ is the vector of unknown control-volume temperatures, and $\mathbf{d}$ is the known right-hand-side vector.

For a mesh containing N control volumes in each direction, the system contains N^2 unknowns. Accordingly, the coefficient matrix has dimensions $N^2 \times N^2$, while both vectors contain N^2 entries.

The cell indices are defined as $i, j = 0, \ldots, N-1$, with i increasing from left to right and j increasing from bottom to top. The physical boundary temperatures are

$$T_{\text{bottom}} = T_{\text{left}} = T_{\text{right}} = 0, \tag{21}$$

and

$$T_{\text{top},i} = \sin\left(\pi \frac{i + \frac{1}{2}}{N}\right). \tag{22}$$

For interior control volumes, the discretized Laplace equation retains the standard five-point stencil:

$$\begin{aligned} T_{i-1,j} + T_{i+1,j} + T_{i,j-1} + T_{i,j+1} \\ - 4T_{i,j} = 0. \end{aligned} \tag{23}$$

For control volumes adjacent to a physical boundary, the ghost-cell temperatures are eliminated using Equation 7. Consequently, the resulting equation depends on the location of the control volume.

For example, a control volume on the left boundary, excluding the corners, satisfies

$$T_{1,j} + T_{0,j-1} + T_{0,j+1} - 5T_{0,j} = -2T_{\text{left}}. \tag{24}$$

Similarly, a corner control volume adjacent to two physical boundaries satisfies an equation of the form

$$T_{\text{nb},1} + T_{\text{nb},2} - 6T_{i,j} = -2T_{b,1} - 2T_{b,2}, \tag{25}$$

where $T_{\text{nb},1}$ and $T_{\text{nb},2}$ are the temperatures of the two neighboring interior control volumes, and $T_{b,1}$ and $T_{b,2}$ are the prescribed temperatures at the adjoining boundaries.

Thus, the central coefficients are -4 for interior control volumes, -5 for non-corner boundary control volumes, and -6 for corner control volumes. The corresponding right-hand-side entries account for the prescribed wall temperatures. The equations for the remaining boundary locations follow from the same procedure, with the spatially varying temperature on the upper boundary evaluated at each cell-center coordinate.

4.7.2 Coefficient Matrix Assembly

To represent the two-dimensional temperature field as a one-dimensional vector, the grid indices are mapped to a linear index using

$$p(i, j) = jN + i, \tag{26}$$

such that the temperature of control volume (i, j) corresponds to entry $p(i, j)$ of $\mathbf{X}$.

Each control volume contributes one row to the coefficient matrix and one entry to the right-hand-side vector. The matrix is initially set to zero, and each row is assembled from the discrete governing equation.

The assembly procedure begins by assigning the central coefficient -4 to the corresponding diagonal entry. For each of the four neighboring directions, the following procedure is applied:

- If the neighboring control volume lies inside the computational domain, its corresponding matrix coefficient is assigned the value $+1$.
- If the neighboring location lies outside the domain, the ghost-cell value is eliminated using the corresponding Dirichlet boundary condition. The central coefficient is reduced by one, and the right-hand-side entry is reduced by twice the prescribed boundary temperature.

For example, eliminating one ghost-cell temperature changes the central coefficient from -4 to -5, while eliminating two ghost-cell temperatures changes it to -6. This procedure automatically accounts for interior, boundary, and corner control volumes without requiring separate matrix-assembly routines for each case.

4.7.3 Direct Solution and Computational Time

After assembling the coefficient matrix and the right-hand-side vector, the system is solved using NumPy's `numpy.linalg.solve()` function:

$$\mathbf{X} = \text{solve}(\mathbf{A}, \mathbf{d}). \tag{27}$$

The resulting vector is reshaped into the two-dimensional temperature field. The solution is then stored in the interior of an array containing an additional layer of ghost cells, and the boundary conditions are applied to reconstruct the corresponding ghost-cell values.

Unlike the iterative methods, the direct method does not require repeated temperature updates or an iterative convergence criterion. Instead, the residual norm can be evaluated after the solution has been obtained to check the consistency of the computed temperature field with the discretized governing equation.

The computational time is measured from the beginning of matrix assembly until the solution and temperature field have been reconstructed. It therefore includes matrix construction, the direct linear solve, and the associated solution reconstruction. The resulting CPU times are compared with those of the iterative methods in Figure 9 and Table 5.

The present implementation stores the coefficient matrix as a dense array, even though most

of its entries are zero. Consequently, its memory requirements and computational cost increase rapidly as the mesh is refined. This provides an important consideration when comparing the direct method with the iterative solvers.

5. Computational Implementation

The numerical solver was implemented in Python using a modular programming structure. Rather than organizing the entire computation in a single script, the project separates numerical algorithms, shared computational utilities, study definitions, and post-processing into independent modules. This organization improves code readability, facilitates debugging, and allows additional numerical methods and verification studies to be incorporated without restructuring the existing implementation.

The project is organized into three main components: the `src` package, which contains the numerical methods and supporting functions; the `studies` package, which defines and executes computational experiments; and the `main.py` script, which acts as the entry point for running the selected studies.

5.1 Software Architecture

The `src` package contains the modules responsible for initializing the temperature field, applying boundary conditions, evaluating residuals, generating the computational grid, defining the analytical solution, and visualizing the results. The numerical algorithms are implemented separately in the solver package.

Table 1 summarizes the principal modules and their responsibilities.

Table 1: Principal modules of the numerical solver.

Module	Purpose
`config.py`	Stores computational settings used by the numerical implementation.
`initialization.py`	Constructs the initial temperature field, including the ghost-cell region.
`boundary_condition.py`	Updates ghost-cell temperatures to enforce the prescribed Dirichlet boundary conditions.
`residual.py`	Evaluates the residual norm for monitoring the convergence of the iterative solvers and assessing the direct solution.
`grid.py`	Generates the structured computational grid and cell-center coordinates.
`exact_solution.py`	Evaluates the analytical temperature distribution for verification.
`solvers/gauss_seidel.py`	Implements the Point Gauss-Seidel iterative method.
`solvers/sor.py`	Implements the point-wise Successive Over-Relaxation method.
`solvers/line_sor.py`	Implements Line-SOR by solving for temperature corrections along vertical lines.
`solvers/implicit.py`	Assembles and directly solves the linear system for the fully implicit method.
`visualization.py`	Generates temperature contours, error fields, line-cut comparisons, and solver convergence and performance plots.
`system_info.py`	Collects computational environment information for documenting the implementation.

5.2 Study Organization

The `studies` package organizes the numerical experiments independently of the solver implementations. Each study defines the mesh sizes, convergence tolerance, relaxation factors, and solver combinations required for a particular investigation. It then executes the relevant solvers and collects the results for tabulation and visualization.

The principal study modules are summarized in Table 2.

Table 2: Computational studies implemented in the project.

Study module	Purpose
`gauss_seidel_verification.py`	Compares the Point Gauss-Seidel solution with the analytical solution and evaluates the numerical error.
`gauss_seidel_mesh_size_study.py`	Investigates the effect of mesh resolution on the convergence behavior and computational cost of Point Gauss-Seidel.
`gs_sor_comparison.py`	Compares Point Gauss-Seidel and SOR for different relaxation factors using a common mesh and convergence tolerance.
`line_sor_comparison.py`	Compares the convergence and computational cost of point-wise SOR and Line-SOR.
`implicit_study.py`	Compares SOR, Line-SOR, and the fully implicit direct method across multiple mesh sizes and verifies the direct solution against the analytical temperature field.

The separation between solver and study modules ensures that the numerical algorithms remain independent of the particular experiments in which they are used. For example, the SOR

solver can be employed with different relaxation factors or mesh sizes without modifying its implementation. Similarly, the study modules can compare multiple solvers while using the same residual evaluation and visualization functions.

5.3 Solution Algorithm

The overall computational procedure depends on the selected study and numerical method. The general workflow consists of the following steps:

1. Specify the computational parameters, including mesh resolution, convergence tolerance, and relaxation factor where applicable.
2. Initialize the temperature field and construct the ghost-cell region required to impose the Dirichlet boundary conditions.
3. Execute the selected numerical solver:
   (a) Point Gauss-Seidel updates the interior control-volume temperatures sequentially.
   (b) SOR performs the same point-wise update with an additional relaxation step.
   (c) Line-SOR solves a linear system for the temperature corrections along each vertical line and applies the relaxation factor.
   (d) The fully implicit direct method assembles the coefficient matrix and right-hand-side vector and solves the resulting linear system.
4. For the iterative methods, update the ghost-cell values and evaluate the residual norm after each complete sweep. Repeat the procedure until the prescribed convergence tolerance is satisfied. For the direct method, reconstruct the temperature field and evaluate the residual after solving the linear system.
5. Record the relevant computational information, including iteration counts, residual histories, and elapsed CPU time where applicable.
6. Generate the required plots and tables to examine numerical accuracy, convergence behavior, and computational performance.

5.4 Solver Implementation

The Point Gauss-Seidel and SOR solvers use the discrete Laplace equation described in Section 4. Point Gauss-Seidel updates each interior temperature sequentially according to Equation 11, while SOR modifies this update using the relaxation factor according to Equation 12. After each complete sweep, the ghost-cell values are updated, and the residual norm is evaluated to determine whether the convergence criterion has been satisfied.

The Line-SOR solver extends the point-wise relaxation approach by solving for the temperature corrections along an entire vertical line. The line system is assembled using the current temperature field, solved for the correction vector, and followed by a relaxed temperature update. The coefficient matrix is constructed once for a given mesh resolution and reused throughout the iterative procedure. In the present implementation, the line system is solved using `numpy.linalg.solve()`.

The fully implicit direct solver follows a different computational procedure. The discrete equations for all control volumes are assembled into a dense coefficient matrix and a right-hand-side vector. The resulting system is solved using `numpy.linalg.solve()`, after which the solution vector is reconstructed as a two-dimensional temperature field. Since the direct method does not rely on iterative temperature updates, it does not produce an iterative convergence history or an iteration count comparable to those of the other methods.

For each solver, the study modules are responsible for measuring and recording the required performance quantities. In particular, the recorded CPU time for the direct method includes matrix assembly and solution, whereas the iterative methods record elapsed computation time over their iterative solution procedures.

5.5 Grid Generation

A structured Cartesian computational mesh is generated by constructing one-dimensional arrays containing the coordinates of the control-volume centers in each spatial direction. These arrays are converted into two-dimensional coordinate matrices using NumPy's `meshgrid()` function with the `indexing='ij'` option.

The resulting coordinate matrices satisfy

$$X[i, j] = x_i, \qquad Y[i, j] = y_j.$$

This indexing convention preserves the matrix arrangement used by the numerical solvers, such that the temperature field $T[i, j]$ corresponds to the physical location (x_i, y_j). The consistent arrangement of the coordinate and temperature arrays simplifies the comparison between numerical and analytical solutions during post-processing.

5.6 Post-Processing and Performance Evaluation

The visualization module provides the functions required to examine the temperature field, numerical error, convergence behavior, and computational cost.

Contour plots are used to compare the numerical and analytical temperature distributions. Error-field plots illustrate the spatial variation of the difference between the two solutions, while horizontal and vertical line cuts provide comparisons of their temperature profiles at selected locations.

The convergence plots display the residual norm as a function of iteration number and CPU time, allowing the iterative performance of Point Gauss-Seidel, SOR, and Line-SOR to be compared. Solver-comparison studies additionally report iteration counts, final residuals, and CPU times in tabular form.

For the fully implicit direct method, verification against the analytical solution is performed independently of iterative convergence monitoring. Its CPU time and final residual are reported alongside those of the iterative methods, enabling a comparison of the complete computational approaches over the tested mesh sizes.

Finally, `main.py` acts as the entry point for executing the selected study functions. This arrangement keeps the main script concise and allows individual verification, mesh-size, and solver-

comparison studies to be run without modifying the numerical algorithms themselves.

6. Results and Discussion

6.1 Temperature Distribution

Figure 3 compares the numerical temperature distribution obtained using the Point Gauss-Seidel solver with the analytical solution of the Laplace equation. The two temperature fields exhibit close agreement throughout the computational domain. The similarity between the numerical and analytical contours supports the correctness of the governing equation, spatial discretization, and boundary-condition implementation.

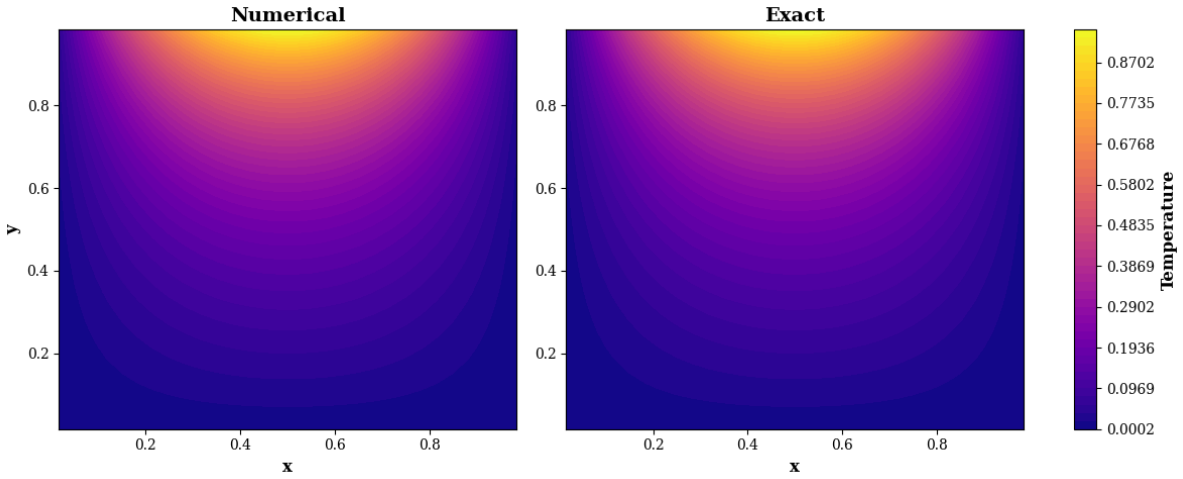


Figure 3: Comparison of the numerical and analytical temperature distributions obtained using the Point Gauss-Seidel solver.

6.2 Error Field

The pointwise difference between the numerical and analytical temperature distributions provides a quantitative assessment of the numerical error. Figure 4 shows the spatial distribution of this difference.

The error field illustrates the local discrepancies between the computed and analytical solutions. A quantitative assessment of the spatial accuracy can additionally be obtained by evaluating error norms and examining their variation across different mesh resolutions.

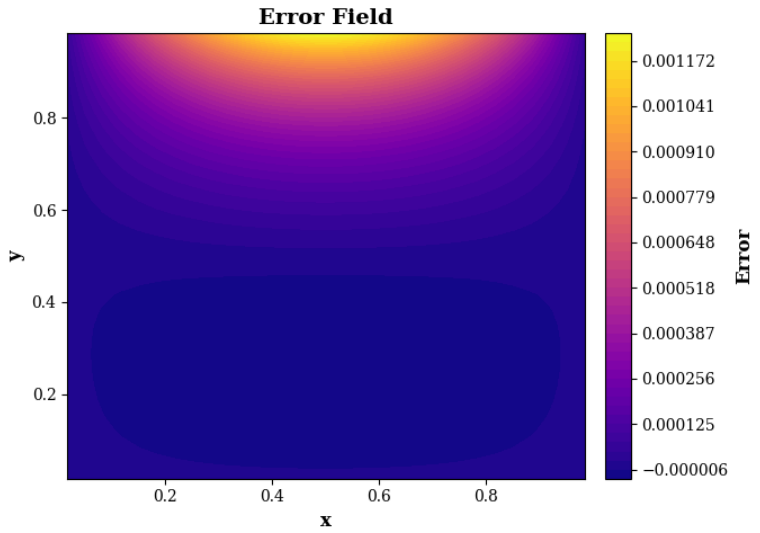


Figure 4: Spatial distribution of the error between the numerical and analytical temperature fields obtained using Point Gauss-Seidel.

6.3 Convergence History

The convergence behavior of the Point Gauss-Seidel solver was monitored using the L_2 residual norm. Figure 5 presents the residual history as a function of both iteration number and CPU time.

The residual decreases toward the prescribed tolerance, indicating convergence toward the solution of the discretized governing equation. The residual-time history also provides a baseline for evaluating the computational performance of the accelerated iterative methods considered in the following subsections.

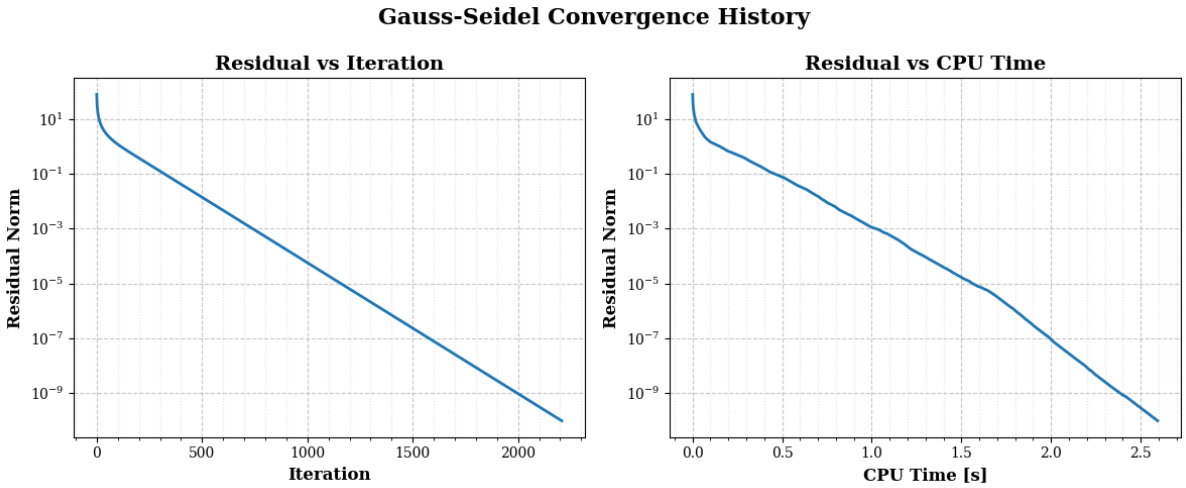


Figure 5: Residual convergence histories of the Point Gauss-Seidel solver as functions of iteration count and CPU time.

6.4 Line-Cut Comparison

To further assess the numerical solution, horizontal and vertical temperature profiles were extracted at selected locations within the computational domain and compared with the corresponding analytical profiles.

As shown in Figure 6, the numerical and analytical temperature profiles exhibit close agreement along the selected cuts. This comparison provides an additional assessment of the spatial variation of the computed temperature field.

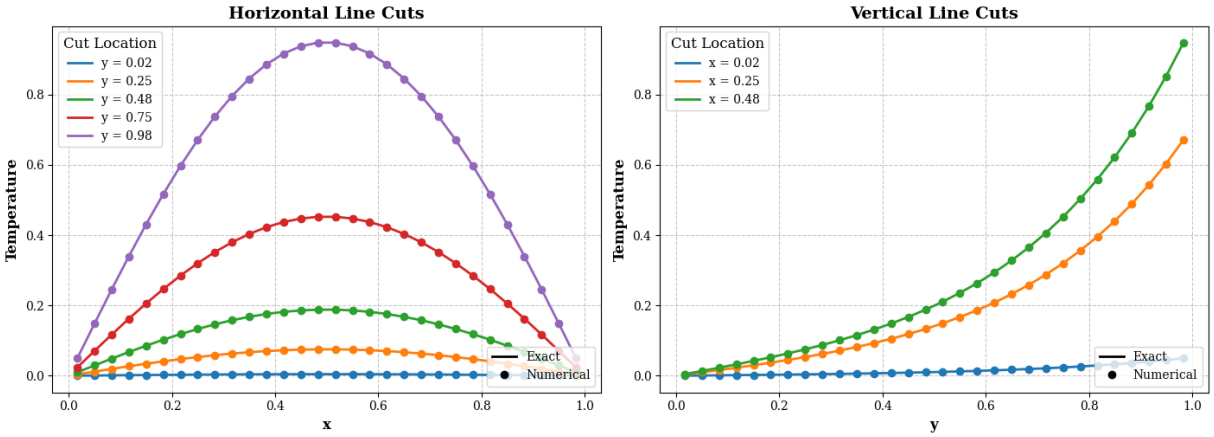


Figure 6: Comparison of numerical and analytical temperature profiles along selected horizontal and vertical line cuts.

6.5 SOR Convergence and Computational Cost

The effect of the relaxation factor on convergence was investigated using the Successive Over-Relaxation (SOR) method. The Point Gauss-Seidel solver and SOR with relaxation factors $\omega = 1.1$, 1.3, and 1.5 were evaluated on a 40×40 mesh with a residual tolerance of 10^{-12}.

Figure 7 compares the residual histories as functions of iteration number and CPU time. Increasing the relaxation factor from 1.1 to 1.5 reduces the iteration count from 3823 to 1549. In the recorded runs, SOR with $\omega = 1.3$ and 1.5 also requires less CPU time than the Point Gauss-Seidel solver. However, the results for $\omega = 1.1$ show that reducing the iteration count does not necessarily reduce the total computation time.

The corresponding iteration counts, CPU times, and final residuals are reported in Table 3.

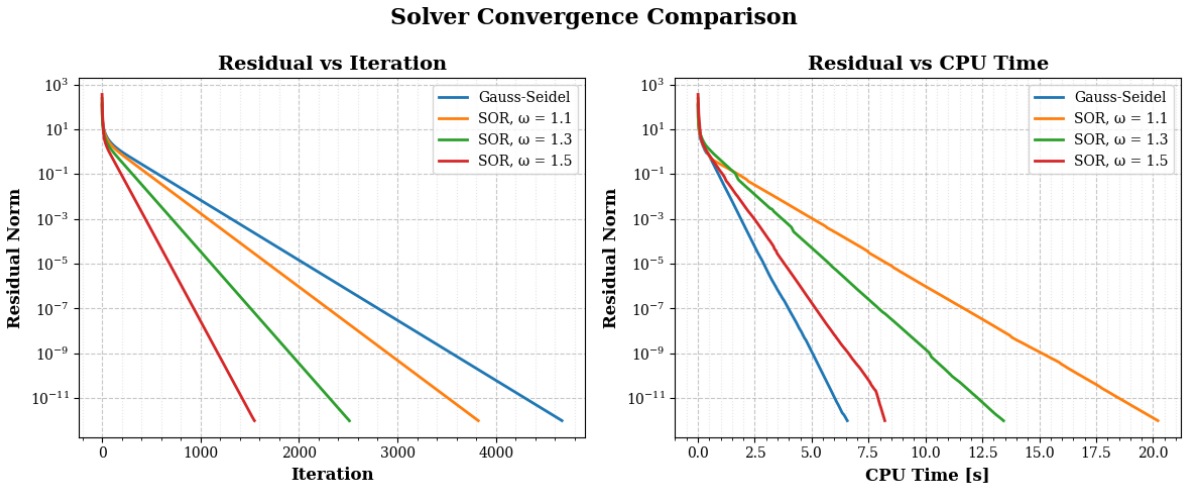


Figure 7: Residual convergence histories of Point Gauss-Seidel and SOR with different relaxation factors on a 40×40 mesh. The panels show residual versus iteration count and residual versus CPU time.

Table 3: Convergence and computational performance of Point Gauss-Seidel and SOR on a 40×40 mesh.

Solver	ω	Iterations	CPU Time [s]	Final Residual
Gauss-Seidel	–	4672	6.1124	9.94×10^{-13}
SOR	1.1	3823	7.7521	9.93×10^{-13}
SOR	1.3	2514	5.4302	9.84×10^{-13}
SOR	1.5	1549	3.2679	9.92×10^{-13}

6.6 Line-SOR Convergence and Computational Cost

The Line-SOR method was compared with point-wise SOR using $\omega = 1.1$ on a 40×40 mesh and with the same residual tolerance of 10^{-12}.

Figure 8 presents the residual histories of the two methods as functions of iteration number and CPU time. Line-SOR requires 1909 iterations, compared with 3823 iterations for point-wise SOR. The corresponding CPU times are 2.2451 s and 8.0495 s, respectively. In this benchmark, Line-SOR therefore achieves convergence in approximately half as many iterations and reduces the recorded CPU time by approximately 72%.

The comparison is summarized in Table 4. These results illustrate the benefit of solving for temperature corrections along an entire line rather than updating each control volume independently.

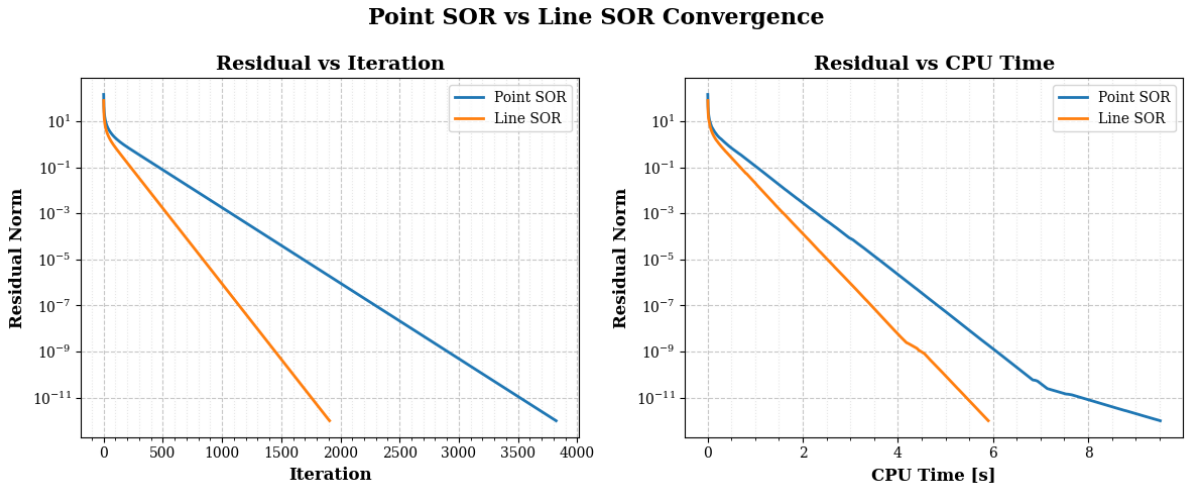


Figure 8: Residual convergence histories of Point-SOR and Line-SOR with $\omega = 1.1$ on a 40×40 mesh. The panels show residual versus iteration count and residual versus CPU time.

Table 4: Convergence and computational performance of Point-SOR and Line-SOR with $\omega = 1.1$ on a 40×40 mesh.

Solver	ω	Iterations	CPU Time [s]	Final Residual
Point SOR	1.1	3823	8.0495	9.93×10^{-13}
Line SOR	1.1	1909	2.2451	1.00×10^{-12}

6.7 Fully Implicit Direct Solver

The fully implicit direct method assembles the discretized equations into a single linear system and solves for the complete temperature field. Unlike the iterative methods, it does not require repeated temperature updates to reach a prescribed residual tolerance.

6.7.1 CPU-Time Comparison Across Mesh Sizes

The computational performance of SOR, Line-SOR, and the fully implicit direct method was evaluated for mesh sizes $N = 10$, 20, 30, and 40. The iterative methods were tested using relaxation factors $\omega = 1.3$ and 1.5, while the direct method was evaluated on each mesh without an iterative relaxation factor.

Figure 9 compares the CPU times of these methods as the mesh is refined. The corresponding iteration counts, CPU times, and final residuals are reported in Table 5.

For the tested meshes, Line-SOR requires less CPU time than point-wise SOR at the same relaxation factor. The direct method has the lowest recorded CPU time for each of the four mesh sizes. However, the direct solver uses a dense coefficient matrix, and its memory requirements and computational cost are expected to increase rapidly for larger systems. These observations should therefore be interpreted within the tested mesh range.

CPU times were recorded in separate benchmark runs. Small differences between tables for the same mesh and relaxation factor can therefore arise from run-to-run timing variation. Comparisons are best interpreted within each benchmark.

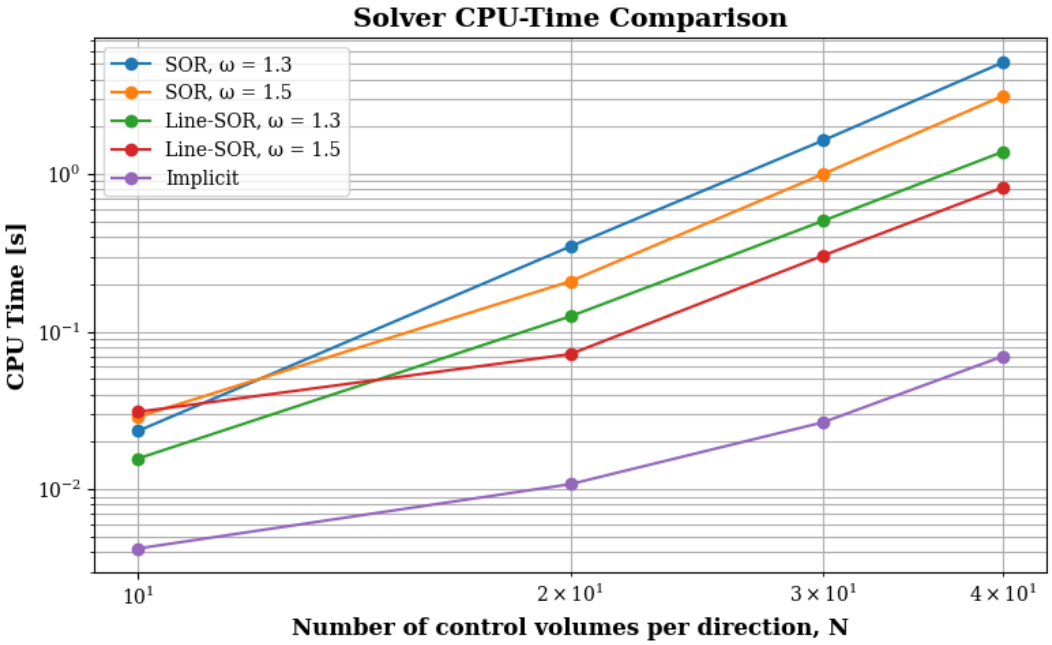


Figure 9: CPU time versus mesh size for SOR, Line-SOR, and the fully implicit direct solver. The iterative methods are shown for $\omega = 1.3$ and 1.5.

Table 5: Iteration counts, CPU times, and final residuals for SOR, Line-SOR, and the fully implicit direct method at different mesh sizes.

N	Solver	ω	Iterations	CPU Time [s]	Final Residual
10	SOR	1.3	146	0.0248	9.86×10^{-13}
10	SOR	1.5	174	0.0319	8.74×10^{-13}
10	Line-SOR	1.3	68	0.0140	9.63×10^{-13}
10	Line-SOR	1.5	135	0.0274	8.44×10^{-13}
10	Implicit	–	Direct	0.0044	2.13×10^{-14}
20	SOR	1.3	622	0.3904	9.63×10^{-13}
20	SOR	1.5	374	0.2044	9.47×10^{-13}
20	Line-SOR	1.3	307	0.1316	9.20×10^{-13}
20	Line-SOR	1.5	176	0.0735	9.04×10^{-13}
20	Implicit	–	Direct	0.0162	7.47×10^{-14}
30	SOR	1.3	1410	1.6450	9.92×10^{-13}
30	SOR	1.5	864	1.0472	9.70×10^{-13}
30	Line-SOR	1.3	700	0.5520	9.71×10^{-13}
30	Line-SOR	1.5	421	0.3309	9.61×10^{-13}
30	Implicit	–	Direct	0.0433	1.73×10^{-13}
40	SOR	1.3	2514	5.2015	9.84×10^{-13}
40	SOR	1.5	1549	3.1708	9.92×10^{-13}
40	Line-SOR	1.3	1250	1.3868	9.92×10^{-13}
40	Line-SOR	1.5	762	0.8401	9.87×10^{-13}
40	Implicit	–	Direct	0.0976	3.50×10^{-13}

6.7.2 Verification Against the Analytical Solution

The fully implicit solution on the 40×40 mesh was compared with the analytical temperature field. Figure 10 shows the numerical and analytical temperature distributions side by side, while Figure 11 presents the spatial distribution of their difference.

For this mesh, the computed L_2 error is 1.6122×10^{-4}, and the maximum absolute error is 7.2049×10^{-4}. These values quantify the difference between the numerical and analytical solutions. The error reflects the spatial discretization and should be distinguished from the residual, which measures how closely the numerical field satisfies the discrete equations.

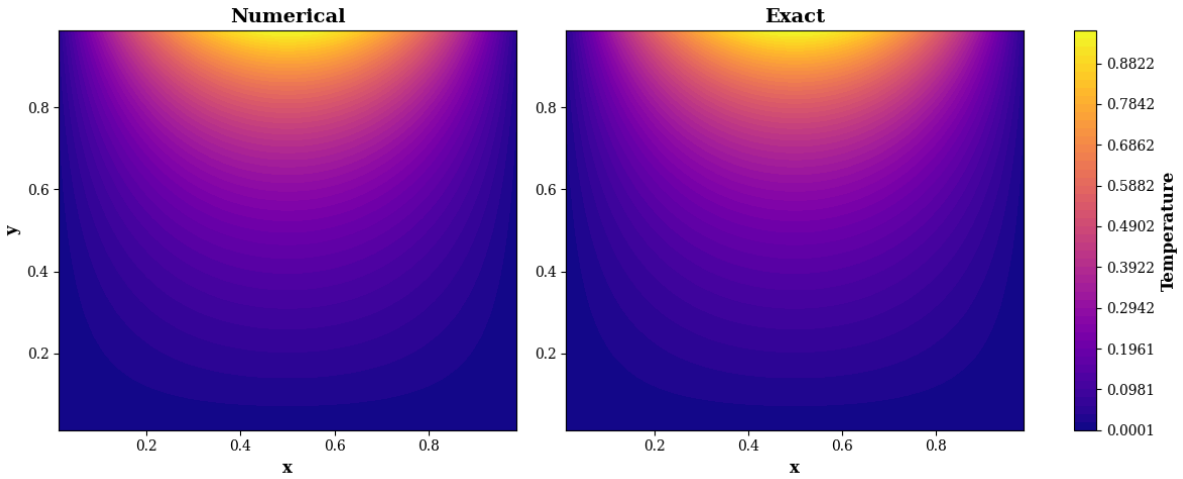


Figure 10: Comparison of the fully implicit numerical temperature distribution and the analytical solution on a 40×40 mesh.

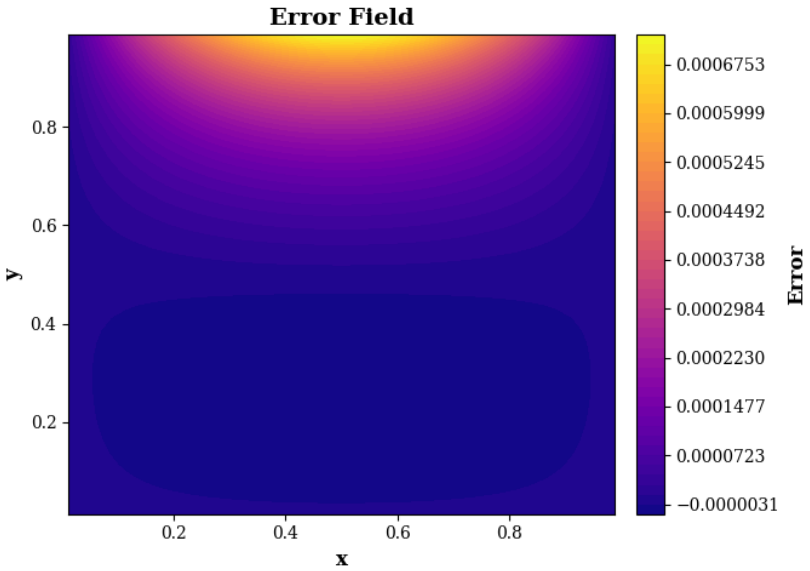


Figure 11: Spatial distribution of the temperature error, defined as the fully implicit numerical solution minus the analytical solution, on a 40×40 mesh.

7. Conclusion

A two-dimensional finite-volume solver for the steady-state heat conduction equation was developed and verified. The Laplace equation was discretized using a second-order cell-centered finite-volume formulation on a structured Cartesian mesh. Four solution methods were implemented: Point Gauss-Seidel, Successive Over-Relaxation (SOR), Line-SOR, and a fully implicit direct method. Dirichlet boundary conditions were imposed through ghost cells, allowing a consistent treatment of the physical boundaries throughout the computational domain.

The numerical solutions were assessed through comparison with the analytical solution using temperature contours, line-cut comparisons, and error distributions. The fully implicit solution on a 40×40 mesh exhibited an L_2 error of 1.6122×10^{-4} and a maximum absolute error of 7.2049×10^{-4}. Residual histories and final residual values were also used to assess the convergence of the numerical methods and their consistency with the discretized governing equation.

The performance studies demonstrated the influence of the solution algorithm and relaxation factor on convergence and computational cost. For the tested 40×40 benchmark with $\omega = 1.1$, Line-SOR required approximately half as many iterations as point-wise SOR and achieved a substantially lower recorded CPU time. The SOR comparisons also showed that increasing the relaxation factor from 1.1 to 1.5 reduced the iteration count, although the computational benefit depended on the selected factor. Across the tested mesh sizes from $N = 10$ to $N = 40$, the fully implicit direct method had the lowest recorded CPU time. However, its dense coefficient matrix imposes increasing memory and computational costs as the problem size grows, limiting the extent to which this result can be generalized to larger meshes.

The modular software architecture separates numerical solvers, shared computational functions, study definitions, and visualization. This organization facilitates systematic comparisons between numerical methods and provides a foundation for further development. Future work may include more extensive mesh-convergence and performance studies, more efficient solution of the tridiagonal systems arising in Line-SOR, and the implementation of additional numerical methods, such as sparse direct solvers and multigrid techniques. The framework can also be extended to more general transport equations encountered in computational fluid dynamics.

Overall, this project establishes a verified computational framework for studying finite-volume discretization and numerical solution strategies. It also provides a basis for further investigating the relationship between numerical accuracy, convergence behavior, and computational efficiency.

A. Computational Setup

All numerical simulations were performed on a single workstation. The hardware configuration is reported to ensure reproducibility of the computational results; however, absolute CPU timing values are system-dependent and should be interpreted accordingly.

A.1 Hardware Configuration

The simulations were executed on the following system:

- Processor: Intel(R) Core(TM) i7-8550U CPU
- Number of physical cores: 4
- Logical cores: 8
- Memory (RAM): 15.88 GB
- Architecture: AMD64

A.2 Software Environment

- Programming language: Python 3.x
- Numerical libraries: NumPy
- Visualization: Matplotlib
- Operating system: Windows

A.3 Timing Methodology

Execution time was measured using a high-resolution wall-clock timer based on `time.perf_counter()`, which provides monotonic timing suitable for performance evaluation of iterative solvers.

Since computational performance depends strongly on hardware architecture, CPU frequency scaling, and system load, timing results are reported for reference only and are not intended for direct cross-machine comparison.